SWIPE TO MASTER IT

SQL Window Functions Masterclass

ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, and frame specifiers — explained from first principles, with real queries and real output.

`ROW_NUMBER()` `RANK()` `DENSE_RANK()` `LAG()` `LEAD()` `NTILE()` `ROWS / RANGE`

Why Every SQL Query Falls Short

Normal SQL is great at two things: filtering rows with **WHERE**, and squashing many rows into one with **GROUP BY**. But real analytics questions rarely fit either box.

"Rank each employee's salary **within their department**." "Show this month's revenue **next to last month's**." "Give me a **running total** without collapsing my rows."

GROUP BY can't do this — it merges rows together and loses the detail. This is exactly the gap **window functions** were built to close.

What Is a Window Function?

A window function performs a calculation **across a set of rows related to the current row** — without collapsing them into a single output row, the way GROUP BY does.

Every row keeps its identity. The function just "looks around" at a window of neighboring rows to compute a rank, a running total, a previous value, or a share of the whole.

Simple analogy: Imagine a leaderboard. Every player still has their own row, but each row also shows their rank compared to everyone else. That's a window function.

Anatomy of the OVER() Clause

Every window function is built from two parts: the **function** itself, and the **OVER()** clause that defines its "window" of rows.

```
SELECT
  employee_id,
  department,
  salary,
  RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS salary_rank
FROM employees;
```

PARTITION BY groups rows into windows. **ORDER BY** sorts rows inside each window. The function runs once per row, using that row's window.

PARTITION BY:
Slicing the Data

PARTITION BY splits your rows into independent groups — one department, one customer, one region at a time — and resets the calculation for each group.

Think of it as a "GROUP BY that doesn't merge rows." Every employee still gets their own row, but the ranking, total, or average only considers rows from their own partition.

> ! Skip PARTITION BY and the function treats the **entire table as one window** — useful sometimes, but easy to do by accident.

ORDER BY Inside the Window

ORDER BY inside OVER() controls the **sequence** rows are processed in for that window. This matters enormously for ranking and offset functions.

For **ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD** — ORDER BY is what defines "first," "next," and "previous." Without it, the order is undefined and results become unreliable.

For aggregate window functions like **SUM() OVER()**, adding ORDER BY also changes the default frame — more on that a few slides ahead.

Window Functions vs GROUP BY

GROUP BY	Window Function
Collapses rows into one per group	Keeps every original row
Can't see individual row detail	Row detail + aggregate, together
One aggregate value per group	Rank, running total, offsets, and more
Used with SELECT + aggregate	Used with the OVER() clause
Filtered with HAVING	Filtered in an outer query

REFERENCE DATASET

The Sample Table
Used In This Guide

Every example ahead uses this same simple **sales** table, so you can follow the logic from one slide to the next.

id	rep	region	month	revenue
1	Asha	East	Jan	42,000
2	Asha	East	Feb	48,500
3	Ravi	West	Jan	51,200
4	Ravi	West	Feb	39,900

A second **employees(department, salary)** table appears later for the ranking examples.

ROW_NUMBER():
The Definition

ROW_NUMBER() assigns a unique, sequential integer to every row in its window — 1, 2, 3, 4 — based on the ORDER BY, with **no repeats, ever**.

Even if two rows tie on the sort column, ROW_NUMBER() still hands out two different numbers. It never skips and never duplicates a number.

This makes it perfect for tasks like **numbering rows, paginating results,** and — most commonly — **removing duplicate records**.

ROW_NUMBER():
A Worked Example

```sql
SELECT rep, month, revenue,
  ROW_NUMBER() OVER (
    PARTITION BY rep
    ORDER BY revenue DESC
  ) AS rn
FROM sales;
```

rep	month	revenue	rn
Asha	Feb	48,500	1
Asha	Jan	42,000	2
Ravi	Jan	51,200	1
Ravi	Feb	39,900	2

ROW_NUMBER(): Killing Duplicates

The #1 real-world use of ROW_NUMBER() is **de-duplication** — keeping only the newest (or best) row per group.

```sql
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY updated_at DESC
    ) AS rn
  FROM customer_records
)
SELECT * FROM ranked WHERE rn = 1;
```

This keeps exactly **one row per customer_id** — the most recently updated one — and discards the rest.

RANK():
The Definition

RANK() also numbers rows by ORDER BY — but tied rows get the **same rank**, and the ranking then **skips** the numbers those ties used up.

If two rows tie for rank 2, the next row jumps straight to rank 4 — there is no rank 3. This mirrors how a real leaderboard or race handles a tie.

Use RANK() whenever ties genuinely matter and you want the gap in numbering to reflect how many rows tied.

RANK():
A Worked Example With Ties

```sql
SELECT department, salary,
  RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS salary_rank
FROM employees;
```

department	salary	salary_rank
Sales	95,000	1
Sales	95,000	1
Sales	88,000	3
Sales	80,000	4

RANK():
Why Rank 2 Disappeared

Look closely at the previous table: two employees tied for **rank 1**, and the very next employee jumped straight to **rank 3**.

That's RANK()'s defining behavior — **tied rows share a rank, and the count of tied rows is "used up"** in the sequence, even though no row actually holds that number.

> ! If you don't want that gap, you want **DENSE_RANK()** instead — covered next.

DENSE_RANK():
The Definition

DENSE_RANK() behaves exactly like RANK() for ties — equal values get equal ranks — but it **never skips a number** afterward.

Where RANK() jumped from 1 straight to 3, DENSE_RANK() would go 1, 1, 2, 3 — a clean, "dense," gap-free sequence of ranks.

This is the right choice when you care about **how many distinct values exist**, not how many individual rows came before.

DENSE_RANK(): A Worked Example

```sql
SELECT department, salary,
  DENSE_RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS dense_rank_val
FROM employees;
```

department	salary	dense_rank_val
Sales	95,000	1
Sales	95,000	1
Sales	88,000	2
Sales	80,000	3

ROW_NUMBER vs RANK vs DENSE_RANK

Function	Ties?	Skips numbers?
ROW_NUMBER()	Never tie — always unique	No skips, no duplicates
RANK()	Tied rows share a rank	Yes — skips after ties
DENSE_RANK()	Tied rows share a rank	No skips — stays dense

Same input, three different outputs — picking the right one depends entirely on how you want ties to be handled.

Top-N Per Group:
The Classic Interview Ask

1

Rank rows inside each group

Apply ROW_NUMBER() or RANK() with PARTITION BY the group column, ORDER BY the metric you care about.

2

Wrap it in a CTE or subquery

Window functions can't be filtered directly in the same SELECT’s WHERE clause.

3

Filter the outer query

Keep only rn <= N (or rank <= N) to get your top N rows per group.

Top-N Per Group: Full Query Example

"Find the top 2 highest-paid employees in every department" — a query almost every SQL interview eventually asks.

```sql
WITH ranked AS (
  SELECT department, name, salary,
    DENSE_RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS rnk
  FROM employees
)
SELECT * FROM ranked WHERE rnk <= 2;
```

LAG():
The Definition

LAG() reaches **backward** and pulls a value from a **previous row** in the same window — without needing a self-join.

By default it looks one row back, but you can pass an offset for "2 rows back," "3 rows back," and so on, plus a **default value** to use when there is no previous row.

LAG() is the standard tool for **period-over-period comparisons**: this month vs last month, today vs yesterday, this order vs the previous order.

LAG():
A Worked Example

```sql
SELECT rep, month, revenue,
  LAG(revenue, 1) OVER (
    PARTITION BY rep
    ORDER BY month
  ) AS prev_month_revenue
FROM sales;
```

rep	month	revenue	prev_month_revenue
Asha	Jan	42,000	NULL
Asha	Feb	48,500	42,000
Ravi	Jan	51,200	NULL
Ravi	Feb	39,900	51,200

LAG():
Offsets and Default Values

LAG() takes up to **three arguments**: the column, how many rows back to look, and what to return when that row doesn't exist.

```
LAG(revenue, 1, 0) OVER (
  PARTITION BY rep ORDER BY month
) AS prev_month_revenue
```

Here, the very first month for each rep — which has no "previous" row — returns **0** instead of NULL, which keeps later math (like growth %) from breaking.

Month-over-Month
Growth, Step by Step

1

Pull the previous period's value

Use LAG() partitioned by the entity, ordered by the time column, to fetch last period's number.

2

Compute the difference

Subtract the LAG() value from the current row's value.

3

Turn it into a percentage

Divide the difference by the previous value (guarding against divide-by-zero) to get growth %.

Month-over-Month Growth Query

```sql
SELECT rep, month, revenue,
  revenue - LAG(revenue) OVER (
    PARTITION BY rep ORDER BY month
  ) AS mom_change
FROM sales;
```

The result shows each rep's revenue **change from the prior month**, sitting right next to the raw numbers — no separate join required.

LEAD():
The Definition

LEAD() is the mirror image of LAG() — it reaches **forward** and pulls a value from a **future row** in the same ordered window.

Same signature as LAG(): column, offset, and an optional default. The only difference is direction — LEAD looks ahead, LAG looks behind.

It's the natural fit whenever the question is framed as **"what happens next"**: next order date, next login, next price change.

LEAD():
A Worked Example

```sql
SELECT rep, month, revenue,
  LEAD(revenue, 1) OVER (
    PARTITION BY rep
    ORDER BY month
  ) AS next_month_revenue
FROM sales;
```

rep	month	revenue	next_month_revenue
Asha	Jan	42,000	48,500
Asha	Feb	48,500	NULL
Ravi	Jan	51,200	39,900
Ravi	Feb	39,900	NULL

LEAD():
Finding Time Between Events

A very common pattern: given a table of **login timestamps** per user, find the gap until each user's **next** login — useful for churn and engagement analysis.

```sql
SELECT user_id, login_at,
  LEAD(login_at) OVER (
    PARTITION BY user_id
    ORDER BY login_at
  ) - login_at AS gap_to_next_login
FROM logins;
```

LAG() vs LEAD():
Same Idea, Opposite Direction

	LAG()	LEAD()
Direction	Looks backward	Looks forward
Typical use	Previous period comparison	Time-to-next-event
First/last row result	First row often NULL	Last row often NULL
Arguments	column, offset, default	column, offset, default

What Is a Window Frame?

PARTITION BY defines the **group** of rows and ORDER BY defines their **sequence** — but the **frame** defines exactly **which rows within that order** the function actually looks at for the current row.

A frame is written as **ROWS BETWEEN ... AND ...** (or RANGE BETWEEN), and it's what makes running totals, moving averages, and "so far" calculations possible.

Without an explicit frame, SQL applies a **default** — which, as we'll see, can quietly produce the wrong answer.

ROWS vs RANGE:
The Key Difference

ROWS counts a fixed number of physical rows before/after the current one — simple and predictable, based on row position.

RANGE groups by the **value** in the ORDER BY column — rows with the **same value** are treated as being in the same position, even if that means including more rows than you expect.

> ! With duplicate order-by values, RANGE can silently include **more rows** than ROWS would — a classic source of subtle bugs.

The Building Blocks of a Frame Clause

UNBOUNDED PRECEDING

Start the window at the very first row of the partition.

N PRECEDING

Start (or end) the window N rows before the current row.

CURRENT ROW

Include the current row itself as a frame boundary.

N FOLLOWING

Extend the window N rows after the current row.

UNBOUNDED FOLLOWING

Extend the window all the way to the last row of the partition.

SUM() OVER(): The Concept

A running total (a.k.a. cumulative sum) adds up a value **row by row**, carrying the running sum forward as it moves down an ordered window.

This is exactly what the frame **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** describes: "everything from the start of the partition, up to and including me."

It's one of the most requested patterns in dashboards: cumulative revenue, cumulative signups, cumulative units shipped.

SUM() OVER(): Query Example

```sql
SELECT rep, month, revenue,
  SUM(revenue) OVER (
    PARTITION BY rep
    ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING
      AND CURRENT ROW
  ) AS running_total
FROM sales;
```

rep	month	revenue	running_total
Asha	Jan	42,000	42,000
Asha	Feb	48,500	90,500

Moving Average: The Concept

A moving average smooths out noisy data by averaging each row together with a fixed number of **neighboring** rows — instead of everything from the start.

A "3-row moving average" uses the frame **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW** — the current row plus the two before it.

Common in finance and analytics dashboards to reveal a **trend** while smoothing out day-to-day spikes.

Moving Average: Query Example

```sql
SELECT day, daily_revenue,
  AVG(daily_revenue) OVER (
    ORDER BY day
    ROWS BETWEEN 2 PRECEDING
      AND CURRENT ROW
  ) AS moving_avg_3d
FROM daily_sales;
```

For every day, this averages **today plus the previous two days** — a rolling 3-day window that slides forward one row at a time.

The Default Frame: A Common Trap

When you add ORDER BY to a window but **omit** the frame clause, SQL doesn't default to "the whole partition" — it defaults to **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**.

That means **SUM(x) OVER (ORDER BY y)** silently becomes a running total, not a total-of-everything — a frequent source of "why is my total wrong?" bugs.

! If you want an aggregate over the **entire partition**, either drop ORDER BY, or write the frame explicitly as **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**.

NTILE():
The Definition

NTILE(n) splits an ordered window into **n roughly equal buckets** and labels each row with its bucket number — 1 through n.

It's the SQL way to compute **quartiles, deciles, or percentiles**: NTILE(4) creates quartiles, NTILE(10) creates deciles, and so on.

If the row count doesn't divide evenly, NTILE() distributes the extra rows to the **earlier** buckets first.

NTILE(): Splitting Customers Into Quartiles

```sql
SELECT customer_id, total_spend,
  NTILE(4) OVER (
    ORDER BY total_spend DESC
  ) AS spend_quartile
FROM customers;
```

Quartile **1** now holds your **top-spending 25%** of customers — ready to filter, segment, or target with a different campaign.

FIRST_VALUE() and LAST_VALUE()

FIRST_VALUE() returns the value from the **first** row of the window frame;
LAST_VALUE() returns the value from the **last** row.

Both take a single column argument and respect PARTITION BY, ORDER BY, and the frame — making them ideal for "compare every row to the group's best (or first) value."

Common use: showing each employee's salary **next to the highest-paid person in their department**, on every row.

FIRST_VALUE(): Query and the Frame Gotcha

```sql
SELECT department, name, salary,
  FIRST_VALUE(salary) OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS top_salary
FROM employees;
```

! LAST_VALUE() is trickier: with the **default frame**, it only sees up to the current row, so it often returns the current row itself. Fix it with **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**.

PERCENT_RANK() and CUME_DIST()

PERCENT_RANK() returns each row's rank as a **percentage between 0 and 1** — "this row is better than X% of the window."

CUME_DIST() (cumulative distribution) returns the **proportion of rows** in the window with a value less than or equal to the current row.

Both are used for **percentile-style reporting** — e.g. "this score is in the 90th percentile" — without manually computing percentiles by hand.

Combining Multiple Window Functions

You can stack as many window functions as you need in a single SELECT — each with its own OVER() clause, even with **different** partitions or orderings.

```sql
SELECT rep, month, revenue,
  RANK() OVER (ORDER BY revenue DESC) AS overall_rank,
  SUM(revenue) OVER (
    PARTITION BY rep ORDER BY month
  ) AS running_total,
  LAG(revenue) OVER (
    PARTITION BY rep ORDER BY month
  ) AS prev_month
FROM sales;
```

COMMON MISTAKE #1

Forgetting ORDER BY Inside OVER()

Ranking and offset functions (**ROW_NUMBER, RANK, LAG, LEAD**) all depend on row order — leave out ORDER BY and the database is free to process rows in **any** order.

The query will still run without an error — it just returns **inconsistent, unreliable results** that may change between runs.

> ! Always pair a ranking or offset function with an explicit **ORDER BY** inside its OVER() clause — no exceptions.

Trusting the Default Frame Blindly

As shown earlier, adding ORDER BY without a frame quietly switches aggregates like SUM() and AVG() into a **running** calculation, not a total-of-everything.

This is one of the most common sources of "the numbers don't match the dashboard" bugs in production SQL.

✓ Be explicit whenever it matters: write out **ROWS BETWEEN ...** instead of relying on the implicit default.

COMMON MISTAKE #3

Filtering a Window Function With WHERE

Window functions are evaluated **after** WHERE and GROUP BY, in the same logical phase as SELECT — so you **cannot** filter directly on a window function's result in the same query.

```
-- This will throw an error:
SELECT *, RANK() OVER (...) AS r
FROM employees
WHERE r = 1; -- r is not defined yet
```

Fix it by wrapping the window function in a **CTE or subquery**, then filtering in the outer query — the same pattern used in the Top-N examples earlier.

Keeping Window Function Queries Fast

1

Index the PARTITION BY / ORDER BY columns

The database can use an existing sort order instead of sorting the data fresh for every partition.

2

Reuse identical OVER() clauses

Some engines compute a shared window definition once and reuse it — use WINDOW clauses where supported.

3

Filter early with WHERE

Reduce the row count before window functions run whenever the filter doesn't depend on the window's own output.

Interview Prep:
Top 3 Paid Per Department

"Write a query that returns the 3 highest-paid employees in each department, including ties."

```sql
WITH ranked AS (
  SELECT department, name, salary,
    DENSE_RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS rnk
  FROM employees
)
SELECT department, name, salary
FROM ranked WHERE rnk <= 3;
```

Notice **DENSE_RANK()** was chosen deliberately — the phrase "including ties" is the exact signal to use it over ROW_NUMBER().

PRACTICE QUESTION

Interview Prep:
Running Total of Daily Revenue

"Given a table of daily revenue, return each day's revenue alongside the running total so far this year."

```sql
SELECT day, revenue,
  SUM(revenue) OVER (
    ORDER BY day
    ROWS BETWEEN UNBOUNDED PRECEDING
      AND CURRENT ROW
  ) AS ytd_running_total
FROM daily_sales
WHERE YEAR(day) = 2026;
```

Cheat Sheet:
Every Function At a Glance

Function	What it returns
ROW_NUMBER()	Unique sequential number, no ties
RANK()	Rank with ties, skips numbers after
DENSE_RANK()	Rank with ties, no gaps
LAG() / LEAD()	Value from a previous / next row
NTILE(n)	Bucket number (1..n) for percentile splits
FIRST_VALUE() / LAST_VALUE()	Value from frame's first / last row
SUM() / AVG() OVER()	Running total / moving average
PERCENT_RANK() / CUME_DIST()	Percentile-style relative position

Now You Can Actually Use Window Functions

ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, running totals, moving averages, and the frame rules that trip everyone up — all in one place.

♥ Like this post

↻ Repost it

+ Follow for more